

ESTUDO DE CASO — O PROBLEMA DA ENTREGA INTELIGENTE

Disciplina: Estruturas de Dados e Análise de Algoritmos

Professor: Alexandre Montanha

Aluno: Lucas Franklin Batista França

Curso: Ciência da Computação

Questão 1 — Classificação do Problema

A) O problema da FastBite pertence à classe NP-Completo.

Esse problema de roteamento envolve crescimento fatorial do número de possibilidades, tornando inviável encontrar solução ótima em tempo polinomial.

O problema pertence à classe NP porque, se uma solução for fornecida, é possível verificar em tempo polinomial se as restrições são respeitadas, como capacidade dos entregadores e tempo máximo de entrega.

Portanto, como o problema pode ser verificado em tempo polinomial e apresenta alta complexidade semelhante a problemas clássicos difíceis, ele é classificado como NP-Completo.

B) O problema da FastBite pode ser reduzido ao **Problema do Caixeiro Viajante (TSP)**.

Para isso, cada restaurante e cliente pode ser tratado como uma **cidade**, e as distâncias entre pontos podem ser tratadas como **custos** entre cidades.

Assim, o objetivo passa a ser encontrar a menor rota que percorra todos os pontos necessários, exatamente como ocorre no TSP. Dessa forma, o problema original pode ser transformado em uma instância equivalente do TSP, demonstrando sua natureza combinatória e sua complexidade.

C) Para demonstrar a inviabilidade da força bruta, podemos considerar **8 pedidos e 3 entregadores**.

Cada pedido pode ser atribuído a um dos **3 entregadores**, resultando em:

$3^8 = 6561$ possíveis atribuições

Supondo uma distribuição de pedidos **3, 3 e 2** para os entregadores, temos:

E1:

$3! = 6$ possíveis rotas

E2:

$3! = 6$ possíveis rotas

E3:

$2! = 2$ possíveis rotas

Total de rotas possíveis:

$$6 \times 6 \times 2 = 72$$

Multiplicando pelas atribuições:

$$6561 \times 72 = 472.392 \text{ combinações possíveis}$$

O que torna inviável testar todas as soluções dentro do limite de tempo exigido pelo sistema.

Questão 2 — Abordagem Gulosa

A) Utilizando os dados do cenário, considere o pedido **P1**, localizado no restaurante **(0,2)**.

As distâncias até os entregadores são calculadas utilizando a **distância Manhattan**, definida por:

$$|x_1 - x_2| + |y_1 - y_2|$$

Assim, temos:

Entregador **E1 (1,1)**:

$$|1 - 0| + |1 - 2| = 1 + 1 = 2$$

Entregador **E2 (5,3)**:

$$|5 - 0| + |3 - 2| = 5 + 1 = 6$$

Entregador **E3 (7,6)**:

$$|7 - 0| + |6 - 2| = 7 + 4 = 11$$

O pedido **P1** é atribuído ao entregador **E1**, pois ele apresenta a menor distância até o restaurante.

Esse procedimento é repetido para todos os pedidos restantes até que todos sejam atribuídos a algum entregador disponível.

Após a etapa de atribuição, cada entregador organiza sua rota escolhendo sempre o **ponto mais próximo** do local atual, repetindo esse processo até que todas as entregas sejam realizadas.

B) O algoritmo é classificado como **guloso (greedy)** porque toma decisões baseadas apenas na melhor opção local em cada etapa.

Nesse caso, a escolha local consiste em selecionar sempre o **entregador mais próximo do restaurante** para cada pedido, sem considerar o impacto futuro dessa decisão sobre o conjunto total de rotas.

Essa estratégia prioriza a melhor solução imediata, mas não garante que o resultado seja o melhor possível em termos globais.

C) Um contraexemplo ocorre quando dois pedidos estão próximos entre si, mas são atribuídos a entregadores diferentes devido a pequenas diferenças iniciais de distância.

Por exemplo, considere dois pedidos próximos entre si, localizados em regiões próximas, e dois entregadores posicionados em locais diferentes. O algoritmo guloso pode atribuir cada pedido a um entregador diferente apenas porque, naquele momento, um deles estava ligeiramente mais próximo de um dos restaurantes.

Entretanto, uma solução global melhor poderia atribuir os dois pedidos ao **mesmo entregador**, permitindo que ele realizasse ambas as entregas em uma única rota contínua. Isso reduziria a distância total percorrida e o tempo final de entrega.

Esse exemplo demonstra que decisões locais nem sempre levam à melhor solução global.

D) A complexidade de tempo do algoritmo depende do número de **pedidos (n)** e do número de **entregadores (m)**.

Para cada pedido, é necessário calcular a distância até todos os entregadores disponíveis. Esse cálculo exige verificar **m entregadores** para cada um dos **n pedidos**.

Essa complexidade é considerada eficiente quando comparada a algoritmos de força bruta, que apresentam crescimento fatorial e se tornam rapidamente inviáveis para grandes quantidades de pedidos.

Questão 3 — Programação Dinâmica e Divisão e Conquista

A) A **Programação Dinâmica (PD)** pode ser aplicada ao problema de roteamento de um único entregador com **k pedidos**, pois esse problema apresenta **subestrutura ótima** e **subproblemas repetidos**.

Nesse contexto, o **subproblema** consiste em determinar a menor rota possível para um **subconjunto de pontos restantes**, considerando a posição atual do entregador e os pontos que ainda precisam ser visitados.

Por exemplo, se um entregador precisa visitar quatro pontos, a Programação Dinâmica pode calcular a melhor rota para visitar subconjuntos menores desses pontos, como conjuntos de dois ou três locais, armazenando os resultados já calculados para reutilizá-los posteriormente. Isso evita recalcular rotas repetidas e torna o processo mais eficiente do que uma busca totalmente exaustiva.

Essa abordagem possui custo aproximado de:

$$O(k^2 \times 2^k)$$

Esse custo ocorre porque é necessário avaliar todos os possíveis subconjuntos de pontos e calcular as melhores rotas entre eles.

Em relação ao uso de memória, essa técnica também exige grande quantidade de armazenamento, pois precisa guardar os resultados intermediários de cada subconjunto analisado.

Para valores pequenos de **k**, como **$k \leq 10$** , ainda é possível utilizar essa técnica em tempo razoável. Entretanto, quando o número de pedidos aumenta, o custo computacional cresce rapidamente, tornando a abordagem **impraticável em tempo real**, principalmente considerando o limite de processamento de poucos segundos exigido pelo sistema.

Portanto, embora a Programação Dinâmica possa produzir soluções de alta qualidade, ela se torna inviável para grandes quantidades de pedidos devido ao seu alto custo computacional e de memória.

B) A estratégia de **Divisão e Conquista** pode ser aplicada ao problema da FastBite dividindo a cidade em **zonas geográficas menores**, como bairros ou quadrantes.

Nesse método, o problema geral é dividido em vários **subproblemas menores e mais simples**, onde cada zona geográfica contém apenas os pedidos e entregadores daquela região. Cada subproblema é resolvido separadamente, reduzindo o tamanho do problema original e tornando o processamento mais rápido.

Essa divisão geográfica permite que diferentes regiões sejam processadas simultaneamente, aumentando a eficiência do sistema e facilitando a escalabilidade quando o número de pedidos aumenta.

Entretanto, essa abordagem apresenta algumas limitações importantes, principalmente nas **fronteiras entre zonas**. Pedidos localizados próximos ao limite entre duas regiões podem ser atribuídos a entregadores diferentes, mesmo que fosse mais eficiente atendê-los por um único entregador localizado na região vizinha.

Isso pode resultar em rotas maiores do que o necessário e em perda de eficiência global.

Portanto, embora a estratégia de Divisão e Conquista ajude a reduzir o tamanho do problema e melhorar o desempenho do sistema, ela precisa ser aplicada com cuidado, especialmente nas regiões de fronteira, para evitar soluções subótimas.

Questão 4 — Comparação das Abordagens

Critério	Greedy	Prog. Dinâmica	Divisão e Conquista
Qualidade da solução	Média	Alta	Média
Complexidade de tempo	Baixa	Alta	Média
Complexidade de espaço	Baixa	Alta	Média
Viabilidade em tempo real	Alta	Baixa	Média
Escalabilidade	Alta	Baixa	Alta
Adaptação a mudanças	Alta	Baixa	Média

Após a análise, observa-se que a abordagem gulosa apresenta o melhor desempenho em tempo real, sendo adequada para aplicações que exigem respostas rápidas. A Programação Dinâmica oferece maior precisão, porém seu alto custo computacional limita seu uso em sistemas grandes. A Divisão e Conquista apresenta boa escalabilidade, mas depende da qualidade da divisão geográfica adotada. Assim, a melhor solução prática consiste em combinar divisão geográfica com heurísticas gulosas, garantindo equilíbrio entre desempenho e qualidade da solução.

Questão 5 — Solução de Engenharia Real

A) Uma **heurística** é um método que busca encontrar soluções suficientemente boas em tempo reduzido, sem garantir necessariamente a solução ótima.

No contexto de algoritmos, heurísticas são utilizadas quando o custo computacional para encontrar a solução ideal é muito alto ou inviável dentro do tempo disponível.

Em sistemas reais, como o da FastBite, heurísticas são preferíveis devido às **restrições de tempo de processamento**. Como o sistema precisa tomar decisões em poucos segundos, tentar calcular a solução ótima seria muito demorado. Dessa forma, utilizar heurísticas permite obter soluções rápidas e com qualidade aceitável, garantindo que o sistema continue funcionando de maneira eficiente.

B) Uma solução prática para o problema da FastBite pode ser estruturada em etapas organizadas, combinando diferentes técnicas para melhorar o desempenho do sistema.

Primeiramente, os pedidos podem ser **divididos por regiões geográficas**, agrupando aqueles que estão próximos entre si. Isso reduz o tamanho dos problemas individuais e facilita o processamento.

Em seguida, pode ser aplicada uma **heurística gulosa** para realizar a atribuição inicial dos pedidos aos entregadores, escolhendo sempre o entregador mais próximo do restaurante.

Depois dessa etapa inicial, pode ser executado um **refinamento local**, no qual pedidos podem ser trocados entre entregadores quando essa troca resultar em melhoria da rota ou redução da distância total percorrida.

Por fim, o sistema deve possuir um **limite máximo de tempo de processamento**, interrompendo os cálculos quando esse tempo for atingido. Isso garante que as decisões sejam tomadas rapidamente e dentro do tempo exigido pelo sistema.

C) Buscar a solução ótima vale a pena quando o número de pedidos é **pequeno**, tornando possível calcular todas as combinações em tempo razoável.

Um exemplo disso ocorre em situações de **entregas programadas com poucos clientes**, como entregas noturnas ou agendadas com antecedência. Nesses casos, como o número de pedidos é reduzido, torna-se viável utilizar algoritmos mais complexos para encontrar a melhor rota possível.

Assim, a busca pela solução ótima é recomendada quando o custo computacional é baixo e o ganho de eficiência justifica o tempo adicional necessário para o cálculo.

Questão 6 — Reflexão Crítica

Em sistemas de larga escala, a solução ótima nem sempre é a melhor escolha. O crescimento exponencial da complexidade torna inviável calcular soluções exatas dentro do tempo disponível.

Assim, a escolha por soluções suficientemente boas representa um equilíbrio entre eficiência computacional e qualidade do resultado. Em sistemas com milhares de pedidos simultâneos, priorizar velocidade e estabilidade é essencial para manter o funcionamento adequado da plataforma.